INTELLIJ IDEA

CHEAT SHEETS

A Complete Reference for IntelliJ IDEA 2026.2

Projects & SDKs · Editing · Live Templates · Navigation · Inspections · Refactoring · Run Configs · Debugger · Testing · Profiler · Maven & Gradle · Spring & Jakarta EE · HTTP Client · Database · Git · AI & Agents

32 REFERENCE SHEETS

CONTENTS

SETUP & INTERFACE

01 Introduction & Installation
02 Projects, Modules & SDKs
03 The Interface
04 Settings & Keymaps
05 Plugins, Themes & Appearance

EDITING & CODE GENERATION

06 Code Completion
07 Live Templates & Surround With
08 Editing Shortcuts & Multi-Caret
09 Generating Code
10 Formatting, Imports & Code Style

NAVIGATION & SEARCH

11 Code Navigation
12 Find Usages & Search
13 Bookmarks, Local History & TODO

ANALYSIS & REFACTORING

14 Inspections & Intentions
15 Rename & Extract Refactorings
16 Change Signature, Move & Inline
17 Structural Search & Migrations

RUN, DEBUG, TEST & PROFILE

18 Run Configurations
19 Debugger Basics
20 Advanced Debugging
21 Testing & Coverage
22 Profiler & JVM Tools

BUILD TOOLS & FRAMEWORKS

23 Maven in IntelliJ
24 Gradle in IntelliJ
25 Spring & Jakarta EE Support
26 HTTP Client & Database Tools

VERSION CONTROL

27 Git in IntelliJ
28 Diff, Merge & History

AI & REMOTE

29 AI Assistant & Agents
30 Remote Development & Collaboration

QUICK REFERENCE

31 Quick Reference: Keyboard Shortcuts
32 Quick Reference: Actions, Templates & Files

IDEA

Table of Contents

32 sheets · syntax · examples · notes

SETUP & INTERFACE 01 – 05

- 01 Introduction & Installat…
- 02 Projects, Modules & S…
- 03 The Interface
- 04 Settings & Keymaps
- 05 Plugins, Themes & App…

EDITING & CODE GENERATION 06 – 10

- 06 Code Completion
- 07 Live Templates & Surro…
- 08 Editing Shortcuts & M…
- 09 Generating Code
- 10 Formatting, Imports & …

NAVIGATION & SEARCH 11 – 13

- 11 Code Navigation
- 12 Find Usages & Search
- 13 Bookmarks, Local Histo…

ANALYSIS & REFACTORING 14 – 17

- 14 Inspections & Intentio…
- 15 Rename & Extract Refa…
- 16 Change Signature, Mov…
- 17 Structural Search & Mi…

RUN, DEBUG, TEST & PROFILE 18 – 22

- 18 Run Configurations
- 19 Debugger Basics
- 20 Advanced Debugging
- 21 Testing & Coverage
- 22 Profiler & JVM Tools

BUILD TOOLS & FRAMEWORKS 23 – 26

- 23 Maven in IntelliJ
- 24 Gradle in IntelliJ
- 25 Spring & Jakarta EE Su…
- 26 HTTP Client & Databas…

VERSION CONTROL 27 – 28

- 27 Git in IntelliJ
- 28 Diff, Merge & History

AI & REMOTE 29 – 30

- 29 AI Assistant & Agents
- 30 Remote Development …

QUICK REFERENCE 31 – 32

- 31 Quick Reference: Keyb…
- 32 Quick Reference: Actio…

Sheet 01

Introduction & Installation

single distribution · Toolbox App · `idea` launcher · EAP builds · other JetBrains IDEs

DESCRIPTION

IntelliJ IDEA is JetBrains' Java IDE. Since 2025.3 it ships as **one distribution**: the core IDE is free, and Ultimate features (Spring, Jakarta EE, the profiler, HTTP Client, full database tools) unlock with a subscription. Install it through the **Toolbox App**, which handles updates, multiple versions and the `idea` command-line launcher.

NOTES

- This set targets **2026.2** (Java 27 day-one support)
- Sheets that need a subscription carry a **Subscription** tag
- PyCharm, WebStorm, Rider share most of this UI and keymap

INSTALL & LAUNCH

TASK	MACOS	WIN / LINUX
Homebrew (macOS)	`brew install --cask intellij-idea`	—
Open a folder from shell	`idea .`	`idea .`
Open file at line	`idea --line 42 Foo.java`	same
Diff two files	`idea diff a.txt b.txt`	same
Search Everywhere	⇧⇧	`Shift Shift`
Find Action	⇧⌘A	`Ctrl+Shift+A`

EXPLAINED

→ Toolbox › ⚙ › **Generate shell scripts** puts `idea` on your PATH

→ EAP builds preview the next release; install side by side in Toolbox

→ **Find Action** runs any command by name, even with no shortcut

→ Help › **Keymap Reference** prints the full keymap PDF

WHERE IN SETTINGS

⚙ `Appearance & Behavior › System Settings › Updates` — channel (Release / EAP)

⚙ `Appearance & Behavior › System Settings` — reopen last project on startup

⚙ `Advanced Settings` — hidden options searchable by name

IN VS CODE

⇄ `code .` ↔ `idea .` ; Command Palette ⇧⌘P ↔ Find Action ⇧⌘A

⇄ VS Code is an editor + extensions; IntelliJ bundles Java tooling · See VS Code 01

FREE VS SUBSCRIPTION (TYPICAL)

AREA	TIER
`Java, Kotlin, Maven, Gradle, Git, debugger`	free
`basic database tools`	free
`Spring, Jakarta EE, app servers`	subscription
`HTTP Client, full Database tools`	subscription
`profiler, remote development`	subscription

VERSION SCHEME

VERSION	MEANING
`2026.2`	second major release of 2026
`2026.2.1`	bug-fix update
`EAP`	early access preview
`Help › About`	shows build and runtime

⚠ COMMON MISTAKES

- Installing outside Toolbox and then via Toolbox — two copies with separate settings and plugins
- Skipping Find Action — most "missing" features are just unbound

✓ TIPS & BEST PRACTICES

- Learn three keys first: ⇧⇧, ⇧⌘A, ⌥↵
- Use Toolbox to roll back if an update breaks a plugin
- Check JetBrains' feature comparison page for the current free/paid split

QUICK SUMMARY

- `single distribution` — free core + subscription
- `Toolbox App` — install & update
- `idea .` — open from shell
- `⇧⌘A` — Find Action

Sheet 02

Projects, Modules & SDKs

open vs import · Project Structure ⌘; · modules · JDK download · language level · .idea/

DESCRIPTION

A **project** is the folder you open; it contains one or more **modules** (usually one per Maven/Gradle module). Each project has an **SDK** (a JDK) and a **language level**. IntelliJ stores its settings in the `.idea/` folder; with Maven or Gradle, the build file is the source of truth and IntelliJ imports from it.

NOTES

- Open the folder with `pom.xml` / `build.gradle` — IntelliJ detects the build tool
- File › Project Structure: `⌘;` / `Ctrl+Alt+Shift+S`
- Download a JDK from the SDK dropdown (Temurin, Corretto, GraalVM…)
- Language level can be lower than the JDK

WHAT TO COMMIT IN .IDEA/

FILE / FOLDER	COMMIT?	HOLDS
`.idea/codeStyles/`	yes	shared code style
`.idea/inspectionProfiles/`	yes	shared inspection profile
`.idea/runConfigurations/`	yes	shared run configs
`.idea/workspace.xml`	no	your window layout, local state
`.idea/shelf/, dataSources.local.xml`	no	personal
`*.iml`	no with Maven/Gradle	module files, regenerated
`.idea/misc.xml`	usually	project JDK name

EXPLAINED

→ The JetBrains `.gitignore` template lists the personal files

→ With Maven/Gradle, change modules in the build file, not Project Structure

→ `misc.xml` stores the JDK **name** — teammates need a JDK with the same name

WHERE IN SETTINGS

⚙ `Project Structure › Project` — SDK and language level

⚙ `Build, Execution, Deployment › Build Tools › Maven / Gradle` — import options

⚙ `File › New Projects Setup › Settings for New Projects` — defaults (IDE-wide)

IN VS CODE

⇄ VS Code has no project model; the Java extension reads Maven/Gradle

⇄ JDK: `java.jdt.ls.java.home` / `java.configuration.runtimes` · See VS Code 24

PROJECT STRUCTURE TABS

TAB	USE
Project	SDK, language level, output dir
Modules	sources, tests, resources roots
Libraries	jars outside a build tool
SDKs	registered JDKs
Artifacts	jars/wars for app servers

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Project Structure	⌘;	Ctrl+Alt+Shift+S
Open project	File › Open	same
Recent projects	File › Recent	same
Close project	File › Close	same

⚠ COMMON MISTAKES

- Committing `workspace.xml` — causes merge conflicts every time someone moves a window
- Editing modules in Project Structure on a Maven project — the next reload overwrites it
- JDK 25 project with language level 17 — new syntax is flagged red even though the JDK supports it

✓ TIPS & BEST PRACTICES

- Use the JetBrains .gitignore template from gitignore.io
- Mark folders via right-click › Mark Directory as › Sources Root
- Use one JDK name across the team (e.g. `temurin-25`)

QUICK SUMMARY

- `project` — the opened folder
- `module` — unit of sources/deps
- `SDK` — JDK + language level
- `.idea/` — commit shared parts only

Sheet 03

The Interface

new UI · main toolbar · tool windows · Search Everywhere ⇧⇧ · Find Action · Distraction-Free

DESCRIPTION

The **new UI** (default since 2024.2) has a compact **main toolbar** (project, branch, run widget), **tool window** stripes on the left, right and bottom, and editor tabs in the middle. Every tool window has a shortcut; **Search Everywhere** finds files, classes, symbols, actions and settings from one box.

NOTES

- Hide all tool windows: `⇧⌘F12` / `Ctrl+Shift+F12`
- `Esc` returns focus to the editor from any tool window
- Drag tool window icons to move them between stripes
- Classic UI is available as a plugin

WINDOW SHORTCUTS

ACTION	MACOS	WIN / LINUX
Search Everywhere	⇧⇧	Shift Shift
Find Action	⇧⌘A	Ctrl+Shift+A
Project tool window	⌘1	Alt+1
Commit tool window	⌘0	Alt+0
Git tool window	⌘9	Alt+9
Problems	⌘6	Alt+6
Terminal	⌥F12	Alt+F12
Recent files	⌘E	Ctrl+E
Switcher	^⇥	Ctrl+Tab
Hide all tool windows	⇧⌘F12	Ctrl+Shift+F12

EXPLAINED

→ `⌘E` then type — filter recent files and tool windows

→ Search Everywhere tabs: All · Classes · Files · Symbols · Actions · Text

→ Press `⇧⇧` twice to include non-project items

→ View › Appearance › **Enter Distraction-Free Mode** hides everything

WHERE IN SETTINGS

⚙ `Appearance & Behavior › Appearance` — theme, UI font, compact mode

⚙ `Appearance & Behavior › Appearance › Tool Window Layout` — save layouts

⚙ `Editor › General › Editor Tabs` — tab placement, limit

IN VS CODE

⇄ Quick Open ⌘P ↔ ⇧⇧; Command Palette ↔ Find Action

⇄ Zen Mode ↔ Distraction-Free Mode · See VS Code 02–03

MAIN TOOLBAR WIDGETS

WIDGET	DOES
`Project`	switch / open projects
`Branch`	Git branches popup
`Run widget`	pick config, run, debug, stop
`⚙ (Settings)`	IDE settings, plugins, updates
`AI icon`	AI chat (if enabled)

SEARCH EVERYWHERE PREFIXES

TYPE	FINDS
/	settings / actions
#	symbols (Java fields, methods)
?	list prefixes
`Foo:42`	file Foo at line 42

⚠ COMMON MISTAKES

- Clicking through menus for actions — ⇧⇧ or ⇧⌘A finds them faster and shows the shortcut
- Closing tool windows to make space — use `⇧⌘F12` to toggle them all
- Losing the editor focus — press `Esc` instead of clicking

✓ TIPS & BEST PRACTICES

- Use `⌘E` as your main file switcher
- Pin a layout with Window › Store Current Layout as Default
- Turn on **Compact mode** on small screens

QUICK SUMMARY

- `⇧⇧` — search everything
- `⌘1 / ⌘9 / ⌘0` — project, Git, commit
- `⌘E` — recent files
- `⇧⌘F12` — hide tool windows

Settings & Keymaps

DESCRIPTION

Settings (⌘, / Ctrl+Alt+S) hold both **IDE-level** options (theme, keymap, plugins) and **project-level** ones (code style, inspections, SDK), marked with a project icon. The **keymap** can follow a preset (macOS, Windows, VS Code, Eclipse) and every action can get a custom shortcut. **Backup and Sync** carries settings between machines.

NOTES

- Search box in Settings matches option names and descriptions
- Project-level settings live in `.idea/` ; IDE-level in your config folder
- `Registry…` (via Find Action) — internal toggles; change with care

STEPS

1. Open Settings: `⌘,` / `Ctrl+Alt+S`
2. Go to **Keymap**; pick a preset (e.g. *macOS*, *VS Code*)
3. Search an action (e.g. *Reformat Code*) or click 🔍 to find by shortcut
4. Right-click › **Add Keyboard Shortcut** and press the keys
5. Resolve conflicts IntelliJ shows, then **Apply**
6. Enable **Backup and Sync** from the ⚙ menu to sync to your JetBrains account

WHERE TO FIND IT

- → Keymap → `Settings › Keymap`
- → Sync → `⚙ › Backup and Sync`
- → Export → `File › Manage IDE Settings › Export Settings`
- → Config dir → `Help › Show Log in Finder` (one level up)

WHERE IN SETTINGS

- ⚙ `Keymap` — preset, custom shortcuts (IDE-wide)
- ⚙ `Appearance & Behavior › System Settings` — autosave, confirm exit
- ⚙ `Editor › General` — smart keys, scrolling, soft wraps

IN VS CODE

⇄ `keybindings.json` ↔ Keymap; VS Code keymap preset exists here

⇄ Settings Sync ↔ Backup and Sync · See VS Code 04–05

SETTINGS SHORTCUTS

ACTION	MACOS	WIN / LINUX
Settings	⌘,	Ctrl+Alt+S
Project Structure	⌘;	Ctrl+Alt+Shift+S
Quick Switch Scheme	^	Ctrl+
Find setting	⇧⌘A, type name	Ctrl+Shift+A

SETTINGS SCOPE

KIND	STORED IN	SHARE BY
IDE	user config dir	Backup and Sync / export
project	.idea/	commit to Git
new-project defaults	IDE config	Settings for New Projects

⚠ COMMON MISTAKES

- Changing code style at IDE level and expecting teammates to get it — project code style must be committed
- Remapping common keys blindly — breaks muscle memory shared with docs and teammates
- Toggling Registry keys from a blog post — can destabilize the IDE

✓ TIPS & BEST PRACTICES

- Quick Switch Scheme (`^\`) toggles theme, keymap, presentation mode
- Search keymap by shortcut to see what a key already does
- Export settings before reinstalling

QUICK SUMMARY

- `⌘,` — Settings
- `Keymap` — presets and custom keys
- `project vs IDE` — where a setting lives
- `Backup and Sync` — roam settings

Sheet 05

Plugins, Themes & Appearance

Marketplace · Installed · bundled plugins · themes · fonts & ligatures · Presentation mode

DESCRIPTION

Plugins add languages, frameworks and tools. Install from **Settings › Plugins › Marketplace**; bundled plugins can be disabled to speed up startup. **Themes** and **editor color schemes** are separate: the theme styles the UI, the scheme colors the code. Editor fonts support ligatures (JetBrains Mono is the default).

NOTES

- Plugins can be required per project: `Settings › Build… › Required Plugins`
- Disable unused bundled plugins (e.g. frameworks you never use)
- Presentation mode enlarges everything for demos
- Zoom editor: `⌘` + mouse wheel if enabled

PLUGIN PICK

+ **Key Promoter X** (Halirutan) — shows the shortcut each time you click a menu
+ A popular theme: **Dracula** or **One Dark** from the Marketplace

STEPS

1. Open `Settings › Plugins` (`⌘,` then type *plugins*)
2. **Marketplace** tab: search, click **Install**, restart if asked
3. **Installed** tab: disable bundled plugins you don't use
4. `Appearance › Theme`: pick a UI theme
5. `Editor › Color Scheme`: pick or duplicate a scheme
6. `Editor › Font`: font, size, line height, **ligatures**

WHERE TO FIND IT

→ Plugins → `Settings › Plugins`

→ Theme → `Appearance & Behavior › Appearance`

→ Font → `Editor › Font`

→ Presentation → `View › Appearance › Enter Presentation Mode`

WHERE IN SETTINGS

⚙ `Settings › Plugins` — install / disable (IDE-wide)

⚙ `Build, Execution, Deployment › Required Plugins` — per project

⚙ `Editor › General › Mouse Control` — Ctrl/⌘+wheel zoom

IN VS CODE

⇄ Extensions view ⇧⌘X ↔ Settings › Plugins

⇄ Themes and fonts work the same way · See VS Code 05, 23

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Settings	⌘,	Ctrl+Alt+S
Quick Switch Scheme	^	Ctrl+
Zoom IDE	Find Action	same
Reset font size	Find Action	same

WORTH DISABLING IF UNUSED

PLUGIN	WHY
Android	large, slows indexing
Kotlin (pure Java teams)	fewer inspections
Framework plugins	fewer gutter icons & checks
Old VCS (SVN, Hg)	less noise

⚠ COMMON MISTAKES

- Installing many overlapping plugins — slow startup and conflicting inspections
- Editing a bundled color scheme directly — duplicate it first, or updates reset it
- Ignoring "Required plugins" prompts — the project's run configs may depend on them

✓ TIPS & BEST PRACTICES

- Use Help › **Diagnostic Tools › Analyze Plugin Startup** to find slow plugins
- Enable ligatures with JetBrains Mono for `->`, `!=`, `>=`
- Keep one dark and one light scheme on Quick Switch

QUICK SUMMARY

- `Marketplace` — install plugins
- `Installed` — disable bundled
- `Theme vs Scheme` — UI vs code colors
- `Presentation mode` — demos

Sheet 06

Code Completion

basic ^Space · type-matching ^⇧Space · postfix (.var, .for, .nn) · ⇧⌘⏎ · parameter info

DESCRIPTION

IntelliJ completes as you type. **Basic completion** lists everything visible; **type-matching** completion filters to values that fit the expected type. **Postfix completion** transforms the expression you just typed (`list.for` → a for loop). **Complete Current Statement** adds the missing `;`, braces or parentheses and moves the caret.

NOTES

- Press basic completion twice for non-imported classes and more
- ⇥ replaces the word under the caret; ⏎ inserts
- Full-line completion is a local AI model, free and offline
- Middle-matching: type `uSer` to find `findUserService`

POSTFIX COMPLETION — TYPE, THEN PRESS ⇥

```
users.for          →  for (User user : users) { }
users.fori         →  for (int i = 0; i < users.size(); i++) { }
findUser(id).var   →  User user = findUser(id);
user.nn            →  if (user != null) { }
user.null          →  if (user == null) { }
name.sout          →  System.out.println(name);
expr.return        →  return expr;
flag.not           →  !flag
list.stream        →  list.stream()
user.cast          →  ((Type) user)
order.try          →  try { order } catch (Exception e) { }
x > 0.if           →  if (x > 0) { }
```

CODE EXPLAINED

- → Postfix works after any expression, including method calls
- → ⇧⌘⏎ — finish the statement: adds `;`, closes `(`, opens `{`
- → ⌘P — show parameter hints for the call under the caret
- → Custom postfix templates: `Editor › General › Postfix Completion`

WHERE IN SETTINGS

- ⚙ `Editor › General › Code Completion` — case sensitivity, auto-popup delay
- ⚙ `Editor › General › Postfix Completion` — enable / add templates
- ⚙ `Editor › General › Inline Completion` — full-line AI completion

IN VS CODE

- ⇄ `^Space` is the same in both editors
- ⇄ No built-in postfix in VS Code; Java extension adds a few · See VS Code 12

COMPLETION SHORTCUTS

ACTION	MACOS	WIN / LINUX
Basic completion	^Space	Ctrl+Space
Type-matching	^⇧Space	Ctrl+Shift+Space
Complete statement	⇧⌘⏎	Ctrl+Shift+Enter
Parameter info	⌘P	Ctrl+P
Quick documentation	F1	Ctrl+Q

POPUP KEYS

KEY	EFFECT
⏎	insert
⇥	replace word
^.	insert + add .
!	insert negated boolean

⚠ COMMON MISTAKES

- Pressing ⏎ in the middle of a word — inserts and leaves the old tail; use ⇥ to replace
- Typing the closing `)` and `;` by hand — ⇧⌘⏎ does it and moves to the next line
- Disabling auto-popup because it's slow — usually a sign indexing hasn't finished

✓ TIPS & BEST PRACTICES

- Use `.var` constantly — it names variables from the type
- Sort suggestions by relevance (⋮ in the popup)
- Type-matching completion after `new` or `return` filters to valid types

QUICK SUMMARY

- `^Space` — basic completion
- `^⇧Space` — type-matching
- `.var / .for / .nn` — postfix
- ⇧⌘⏎ — complete statement

Sheet 07

Live Templates & Surround With

psvm / main · sout · fori / iter · Insert Live Template ⌘J · Surround With ⌥⌘T · variables

DESCRIPTION

Live templates expand an abbreviation into code: type `psvm` and press ⇥. **Surround With** wraps the selection in a construct (`if`, `try/catch`, `for`). You can add your own templates with **variables** (`$NAME$`) and functions that suggest values from context.

NOTES

- `⌘J` lists templates valid at the caret
- Templates are scoped by context (Java statement, expression, comment…)
- Share custom templates by exporting settings

PLUGIN PICK

+ **String Manipulation** (Vojtech Krasa) — case conversions, sorting, escaping on selections

BUILT-IN TEMPLATES AND A CUSTOM ONE

```
psvm / main  →  public static void main(String[] args) { }
sout         →  System.out.println();
soutv        →  System.out.println("name = " + name);
fori         →  for (int i = 0; i < ; i++) { }
iter         →  for (User user : users) { }
ifn          →  if (x == null) { }
thr          →  throw new
prsf         →  private static final

// Custom: Settings › Editor › Live Templates › + › "log"
private static final Logger LOG =
    LoggerFactory.getLogger($CLASS$.class);    // CLASS = className()
```

CODE EXPLAINED

- → `soutv` picks the nearest variable automatically
- → `$CLASS$` with expression `className()` fills the current class
- → Select code, `⌥⌘T`, choose `try / catch` to wrap it
- → `$END$` marks where the caret lands

WHERE IN SETTINGS

- ⚙ `Editor › Live Templates` — add / edit groups (IDE-wide)
- ⚙ `Editor › General › Postfix Completion` — the postfix variant
- ⚙ `Editor › Live Templates › By default expand with` — Tab / Enter / Space

IN VS CODE

- ⇄ User snippets JSON ↔ Live Templates; `${1:name}` ↔ `$NAME$`
- ⇄ Surround With ↔ snippets with `$TM_SELECTED_TEXT` · See VS Code 10

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Insert Live Template	⌘J	Ctrl+J
Surround With	⌥⌘T	Ctrl+Alt+T
Next variable	⇥	Tab

TEMPLATE FUNCTIONS

FUNCTION	RETURNS
`className()`	current class
`methodName()`	current method
`suggestVariableName()`	name from type
`date()`	today
`clipboard()`	clipboard text

⚠ COMMON MISTAKES

- Defining a template without setting its context — it never appears; click **Define** under the template
- Overlapping abbreviations with completion words — templates pop up when you don't want them
- Forgetting `$END$` — caret ends at the last variable instead of the body

✓ TIPS & BEST PRACTICES

- Select code then Tools › **Save as Live Template** to bootstrap one
- Group team templates and export them
- Use Surround With for try-with-resources on a selection

QUICK SUMMARY

- `psvm / sout / iter` — built-ins
- `⌘J` — insert template
- `⌥⌘T` — surround with
- `$VAR$` — template variables

Sheet 08

Editing Shortcuts & Multi-Caret

duplicate ⌘D · delete line ⌘⌫ · move statement · Extend Selection ⌥↑ · multi-caret ^G · paste history

DESCRIPTION

These are the shortcuts that save the most time day to day. **Extend Selection** grows the selection by syntax (word → expression → statement → block). **Move Statement** moves a whole method or statement up and down. **Multiple carets** edit several places at once; select the next occurrence with ^G.

NOTES

- `⌥`-drag (macOS) / `Alt`-drag adds a column selection
- `⌥⇧`-click adds a caret anywhere
- `Esc` collapses back to one caret

EDITING

ACTION	MACOS	WIN / LINUX
Duplicate line / selection	`⌘D`	`Ctrl+D`
Delete line	`⌘⌫`	`Ctrl+Y`
Move statement up / down	`⇧⌘↑` / `⇧⌘↓`	`Ctrl+Shift+↑` / `↓`
Move line up / down	`⌥⇧↑` / `⌥⇧↓`	`Alt+Shift+↑` / `↓`
Extend / shrink selection	`⌥↑` / `⌥↓`	`Ctrl+W` / `Ctrl+Shift+W`
Add caret at next occurrence	`^G`	`Alt+J`
Select all occurrences	`^⌘G`	`Ctrl+Alt+Shift+J`
Unselect occurrence	`^⇧G`	`Alt+Shift+J`
Column selection mode	`⇧⌘8`	`Alt+Shift+Insert`
Comment line / block	`⌘/` / `⌥⌘/`	`Ctrl+/` / `Ctrl+Shift+/`
Paste from history	`⇧⌘V`	`Ctrl+Shift+V`
Join lines	`^⇧J`	`Ctrl+Shift+J`
Toggle case	`⇧⌘U`	`Ctrl+Shift+U`
Undo / redo	`⌘Z` / `⇧⌘Z`	`Ctrl+Z` / `Ctrl+Shift+Z`

EXPLAINED

- → `⌘⌫` / `Ctrl+Y` deletes the line without touching the clipboard
- → Move Statement keeps code valid; Move Line moves text only
- → `^G` repeatedly, then type — rename local occurrences without a refactor
- → Paste from history keeps your last clipboard entries

WHERE IN SETTINGS

- ⚙ `Editor › General › Smart Keys` — auto-close brackets, smart Home
- ⚙ `Editor › General › Clipboard` — history size
- ⚙ `Keymap` — rebind `Ctrl+Y` if you want redo on Windows

IN VS CODE

- ⇄ ⌘D (VS Code) = next occurrence; IntelliJ ^G. ⌘D here duplicates
- ⇄ ⌥↑ moves lines in VS Code; here it extends selection · See VS Code 06–07

SELECTION TRICKS

TRICK	HOW
`select word`	`⌥↑` once
`select string contents`	`⌥↑` until inside quotes
`select method`	keep pressing `⌥↑`
`rectangular block`	`⌥`-drag / `Alt`-drag

⚠ COMMON MISTAKES

- Using `Ctrl+Y` as redo on Windows — it deletes the line in IntelliJ's default keymap
- Hitting `⌘D` for next occurrence (VS Code habit) — duplicates the line instead
- Editing with multiple carets across code you can't see — scroll to check every caret

✓ TIPS & BEST PRACTICES

- Learn `⌥↑` — it replaces most mouse selections
- `⇧⌘↑` reorders methods without cut/paste
- Use **Join Lines** to collapse a declaration and assignment

QUICK SUMMARY

- `⌘D` — duplicate
- `⌥↑` — extend selection
- `^G` — next occurrence caret
- `⇧⌘V` — paste history

Generating Code

DESCRIPTION

Generate (⌘N / Alt+Insert) writes boilerplate: constructors, getters and setters, equals()/hashCode(), toString(), delegate methods and test methods. **Override** and **Implement** methods list what a superclass or interface offers. Intentions (⌥⏎) can convert a class to a **record** when it qualifies.

NOTES

- Generated accessors follow JavaBeans naming (Jakarta EE 05)
- `equals/hashCode` templates: IntelliJ default, Java 7+ `Objects`, Apache, Guava
- In tests, `⌘N` offers Test Method, SetUp, TearDown

PLUGIN PICK

+ **Lombok** (bundled) — needs annotation processing on: `Build… › Compiler › Annotation Processors`; IntelliJ offers to enable it

BEFORE ⌘N → AFTER

```
public class Customer {                    // caret here, ⌘N
    private final String name;
    private int points;
}
// Generate › Constructor, Getter and Setter, equals() and hashCode()
public Customer(String name) { this.name = name; }
public String getName() { return name; }
public int getPoints() { return points; }
public void setPoints(int points) { this.points = points; }
@Override public boolean equals(Object o) {
    if (!(o instanceof Customer c)) return false;
    return points == c.points && Objects.equals(name, c.name);
}
@Override public int hashCode() { return Objects.hash(name, points); }

// ⌥⏎ on the class › "Convert to record" (when fields are final)
public record Customer(String name, int points) { }
```

CODE EXPLAINED

- → `final` fields get no setter option
- → `instanceof` pattern in `equals` — newer templates use it
- → `^O` override / `^I` implement list inherited methods
- → Convert to record needs immutable fields and no extra state

WHERE IN SETTINGS

- ⚙ `Editor › Code Style › Java › Code Generation` — field prefixes, `final` params
- ⚙ `Editor › File and Code Templates` — new file headers
- ⚙ `Build… › Compiler › Annotation Processors` — Lombok

IN VS CODE

- ⇄ Java extension: Source Action › Generate Getters and Setters
- ⇄ Fewer generators; no Convert-to-record intention · See VS Code 13

GENERATE SHORTCUTS

ACTION	MACOS	WIN / LINUX
Generate menu	⌘N	Alt+Insert
Override methods	^O	Ctrl+O
Implement methods	^I	Ctrl+I
Delegate methods	⌘N › Delegate	Alt+Insert › Delegate

GENERATOR OPTIONS

ITEM	OPTIONS
Getter/Setter	template, null checks
equals/hashCode	template, fields, non-null
toString	template (String concat, builder)
Constructor	pick fields, @NotNull params

⚠ COMMON MISTAKES

- Generating `equals` with mutable fields used in a `HashSet` — hash changes after insert
- Regenerating `toString` by hand after adding fields — use Generate again and replace
- Lombok without annotation processing — red code, missing getters at compile time

✓ TIPS & BEST PRACTICES

- Use records for DTOs and value objects; generate only what records can't
- Turn on Code Generation › *Add @Override* everywhere
- In tests, `⌘N › Test Method` creates the method skeleton

QUICK SUMMARY

- `⌘N` — generate
- `^O / ^I` — override / implement
- `Convert to record` — intention
- `Lombok` — enable annotation processing

Sheet 10

Formatting, Imports & Code Style

Reformat Code ⌥⌘L · Optimize Imports ^⌥O · code style schemes · `.editorconfig` · Actions on Save

DESCRIPTION

Reformat Code applies the project's **code style** (indents, wrapping, spacing). **Optimize Imports** removes unused imports and sorts the rest. Both can run automatically with **Actions on Save**. Commit the code style to `.idea/codeStyles/` or use **`.editorconfig`** so everyone formats the same way.

NOTES

- `.editorconfig` overrides IDE code style when supported
- Reformat a selection, a file, or a whole directory (right-click)
- Import on-demand thresholds decide when `*` imports appear

PLUGIN PICK

- **CheckStyle-IDEA** — runs Checkstyle rules in the editor
- **google-java-format** — replaces the formatter with Google style

.EDITORCONFIG FOR A JAVA PROJECT

```
root = true

[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true

[*.java]
indent_style = space
indent_size = 4
max_line_length = 120
ij_java_class_count_to_use_import_on_demand = 99
ij_java_names_count_to_use_import_on_demand = 99
```

CODE EXPLAINED

→ `ij_java_*` keys — IntelliJ-specific settings inside `.editorconfig`

→ `99` thresholds — effectively never use `*` imports

→ Export: `Code Style › ⚙ › Export › EditorConfig File`

→ Reformat on a directory: right-click › Reformat Code

WHERE IN SETTINGS

⚙ `Editor › Code Style › Java` — scheme (Project vs IDE)

⚙ `Tools › Actions on Save` — reformat, optimize imports, rearrange

⚙ `Editor › General › Auto Import` — optimize imports on the fly

IN VS CODE

⇄ Format Document ⇧⌥F ↔ ⌥⌘L; `editor.formatOnSave` ↔ Actions on Save

⇄ `.editorconfig` works in both (extension in VS Code) · See VS Code 14

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Reformat Code	⌥⌘L	Ctrl+Alt+L
Reformat with dialog	⌥⇧⌘L	Ctrl+Alt+Shift+L
Optimize Imports	^⌥O	Ctrl+Alt+O
Auto-indent lines	^⌥I	Ctrl+Alt+I

ACTIONS ON SAVE

OPTION	DOES
Reformat code	whole file or changed lines
Optimize imports	remove unused
Rearrange code	order members
Run code cleanup	apply safe inspections

⚠ COMMON MISTAKES

- Reformatting whole files in a PR — huge diffs; choose **changed lines only** in Actions on Save
- Keeping code style at IDE level — teammates format differently; switch the scheme to **Project**
- Mixing formatter plugins with Actions on Save — they fight over the same lines

✓ TIPS & BEST PRACTICES

- Mark generated code with `// @formatter:off` / `on` (enable in Code Style › Formatter)
- Add `.editorconfig` to the repo root for all editors
- Run Reformat on a directory once, commit it alone, then turn on save actions

QUICK SUMMARY

- `⌥⌘L` — reformat
- `^⌥O` — optimize imports
- `.editorconfig` — shared style
- `Actions on Save` — automate

Code Navigation

DESCRIPTION

Navigation is where IntelliJ pulls furthest ahead of a text editor: it knows every type, so jumps are exact. Go to a **declaration**, its **implementations**, the **super method**, or browse **type** and **call hierarchies**. **Back** and **Forward** retrace your jumps like a browser.

NOTES

- ⌘-click = Go to Declaration
- ⌘B on a declaration shows its usages instead
- Navigate › Related Symbol (^⌘↑) jumps between test and class or interface pairs

NAVIGATION

ACTION	MACOS	WIN / LINUX
Go to Declaration	⌘B	Ctrl+B
Go to Implementation(s)	⌥⌘B	Ctrl+Alt+B
Go to Super Method	⌘U	Ctrl+U
Go to Type Declaration	⇧^B	Ctrl+Shift+B
Type Hierarchy	^H	Ctrl+H
Call Hierarchy	^⌥H	Ctrl+Alt+H
File Structure	⌘F12	Ctrl+F12
Recent Files	⌘E	Ctrl+E
Back / Forward	⌘[/ ⌘]	Ctrl+Alt+← / →
Last Edit Location	⇧⌘⌫	Ctrl+Shift+Backspace
Go to Line:Column	⌘L	Ctrl+G
Next / prev method	^↓ / ^↑	Alt+↓ / ↑

EXPLAINED

- → File Structure: type to filter members; ⌘F12 again shows inherited
- → Call Hierarchy toggles callers ↔ callees
- → Gutter icons (↑ ↓ ●) jump to overridden / implementing methods
- → Back/Forward cross files; Last Edit Location only follows edits

WHERE IN SETTINGS

- ⚙ Editor › General › Breadcrumbs — show path at top or bottom
- ⚙ Editor › General › Code Folding — what folds by default
- ⚙ Keymap › Main Menu › Navigate — every navigation action

IN VS CODE

⇄F12 Go to Definition ↔ ⌘B ; ⇧⌘O Go to Symbol ↔ ⌘F12

⇄Go Back ^- ↔ ⌘[· See VS Code 08

GO TO BY NAME

ACTION	MACOS	WIN / LINUX
Class	⌘O	Ctrl+N
File	⇧⌘O	Ctrl+Shift+N
Symbol	⌥⌘O	Ctrl+Alt+Shift+N
Action	⇧⌘A	Ctrl+Shift+A

⚠ COMMON MISTAKES

- Scrolling to find a method — ⌘F12 and type is faster
- Using text search to find callers — Call Hierarchy is exact and ignores comments
- Forgetting Back — ⌘[returns after a deep jump chain

✓ TIPS & BEST PRACTICES

- CamelHump search: ⌘O then OSI finds OrderServiceImpl
- Add :42 after a file name to open at a line
- Pin the Hierarchy tool window when exploring a large type tree

QUICK SUMMARY

- ⌘B — declaration
- ⌥⌘B — implementations
- ⌘F12 — file structure
- ⌘[/ ⌘] — back / forward

Find Usages & Search

DESCRIPTION

Find Usages searches by meaning: it finds real references to a symbol, not text that happens to match. **Find in Files** is the text search, with regex, file masks and **scopes** (project, module, tests, changed files). Use usages for code and text search for strings, config and comments.

NOTES

- Show Usages opens a popup; Find Usages opens the tool window
- Regex replace supports `$1` groups and case conversion `\U$1`
- Custom scopes: `Settings › Appearance & Behavior › Scopes`

SEARCH

ACTION	MACOS	WIN / LINUX
Find Usages	⌥F7	Alt+F7
Show Usages (popup)	⌥⌘F7	Ctrl+Alt+F7
Highlight usages in file	⇧⌘F7	Ctrl+Shift+F7
Find in file	⌘F	Ctrl+F
Replace in file	⌘R	Ctrl+R
Find in Files	⇧⌘F	Ctrl+Shift+F
Replace in Files	⇧⌘R	Ctrl+Shift+R
Next / prev occurrence	⌘G / ⇧⌘G	F3 / Shift+F3
Select next match	^G	Alt+J

EXPLAINED

→ Find Usages groups results: read / write access, imports, tests

→ `.*` in the search bar toggles regex; `Cc` match case; `W` whole word

→ File mask (`*.yml`) limits Find in Files

→ Open results in the Find tool window to keep them

WHERE IN SETTINGS

⚙ `Appearance & Behavior › Scopes` — named scopes for search and inspections

⚙ `Editor › General › Smart Keys` — Find selected text with ⌘F

⚙ `Advanced Settings › Find in Files` — results limit

IN VS CODE

⇄ Find All References ⇧F12 ↔ `⌥F7` ; Search view ⇧⌘F ↔ Find in Files

⇄ Search Editor ↔ Find tool window · See VS Code 09

REGEX REPLACE EXAMPLES

FIND	REPLACE	EFFECT
`get(\w+)\(\)`	$1	strip get prefixes
`(\w+)_(\w)`	$1\U$2	snake → camel
`javax\.(\w+)`	jakarta.$1	namespace move

SCOPES

SCOPE	SEARCHES
`Project Files`	all project files
`Production / Test`	main or test sources
`Changed Files`	uncommitted changes
`Custom`	pattern you define

⚠ COMMON MISTAKES

- Text-searching a method name to rename it — misses overloads and hits comments; use Rename (sheet 15)
- Forgetting the scope is "Directory" from last time — results look incomplete
- Regex replace without previewing — use **Preview** before Replace All

✓ TIPS & BEST PRACTICES

- `⌥⌘F7` popup is fastest for a quick look
- Use **Changed Files** scope before committing
- Structural Search (sheet 17) for code-shape searches

QUICK SUMMARY

- `⌥F7` — semantic usages
- `⇧⌘F` — text search
- `regex` — `$1` groups
- `scope` — narrow results

Sheet 13

Bookmarks, Local History & TODO

bookmarks F3 · mnemonic ⌥F3 · Local History · TODO tool window · Recent Locations ⇧⌘E · scratch files

DESCRIPTION

Bookmarks mark lines you return to; mnemonic bookmarks get a letter or digit. **Local History** is IntelliJ's own versioning: every change is kept locally, so you can restore deleted code even without Git. The **TODO** tool window collects `TODO`/`FIXME` comments, and **scratch files** give you a runnable throwaway file.

NOTES

- Local History is per machine; it's not a backup
- Custom TODO patterns: `Editor › TODO`
- Scratch files live outside the project

PLUGIN PICK

+ **Rainbow Brackets** (izhangzhihao) — colors matching brackets and indents for quick scanning

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Toggle bookmark	`F3`	`F11`
Mnemonic bookmark	`⌥F3`	`Ctrl+F11`
Show bookmarks	`⌘F3`	`Shift+F11`
Jump to mnemonic 1	`^1`	`Ctrl+1`
Recent Locations	`⇧⌘E`	`Ctrl+Shift+E`
New scratch file	`⇧⌘N`	`Ctrl+Alt+Shift+Insert`
Local History	`Find Action › Local History`	same

EXPLAINED

→ Right-click a file or folder › **Local History › Show History**

→ Select code › Local History › **Show History for Selection**

→ Put a **label** in Local History before risky changes

→ Scratch files can be Java, SQL, JSON, HTTP…

WHERE IN SETTINGS

⚙ `Editor › TODO` — patterns and filters

⚙ `Advanced Settings › Local History` — how many days to keep

⚙ `Editor › General › Gutter Icons` — bookmark icons

IN VS CODE

⇄ Bookmarks need an extension; Timeline view ≈ Local History

⇄ Todo Tree extension ↔ TODO tool window · See VS Code 09

RECOVERY OPTIONS

LOST…	USE
`deleted file`	Local History on the parent folder
`overwritten code`	Show History for Selection
`uncommitted work after reset`	Local History, then revert
`committed work`	Git reflog / Git tool window

TODO PATTERNS

PATTERN	MATCHES
`\btodo\b.*`	TODO comments
`\bfixme\b.*`	FIXME comments
`custom`	e.g. `\bperf\b.*`

⚠ COMMON MISTAKES

- Relying on Local History as a backup — it's local, time-limited and lost if caches are cleared
- Leaving TODOs with no owner — the tool window fills up with noise
- Saving scratch work in the project and committing it by accident

✓ TIPS & BEST PRACTICES

- Recent Locations shows code snippets around each visit — great after a long jump chain
- Filter the TODO window by **Current File** or **Changelist**
- Use scratch files with the HTTP client or SQL console for quick experiments

QUICK SUMMARY

- `F3` — bookmark
- `Local History` — restore anything recent
- `TODO` — tracked comments
- `⇧⌘N` — scratch file

Inspections & Intentions

DESCRIPTION

Inspections analyze code as you type and highlight bugs, smells and modernizations. **Intentions** are quick fixes and conversions offered at the caret via **⌥⏎** (Alt+Enter), the most useful key in the IDE. Inspection **profiles** set severities and can be shared with the project. **Inspect Code** runs everything across a scope.

NOTES

- Yellow = warning, red = error, underline = weak warning
- Lightbulb 💡 = quick fix; pencil = intention
- `F2` jumps to the next problem

PLUGIN PICK

+ **SonarQube for IDE** (SonarSource) — Sonar rules in the editor; connect to a server to share rule sets

⌥↵ ON HIGHLIGHTED CODE

```
List<String> names = new ArrayList<String>(); // ⌥⏎ › use <>
for (int i = 0; i < users.size(); i++) {      // ⌥⏎ › for-each
    names.add(users.get(i).getName());
}                                             // ⌥⏎ › stream
if (obj instanceof User) {                    // ⌥⏎ › pattern
    User u = (User) obj;
}
String s = "Hello " + name + "!";             // ⌥⏎ › String.format
@SuppressWarnings("unchecked")                // ⌥⏎ › suppress
List<User> raw = (List<User>) cache.get(key);
// noinspection ConstantConditions            // one statement
```

CODE EXPLAINED

→ `⌥⏎` › arrow right → fix all in file / suppress / edit inspection

→ Inspection names work in `@SuppressWarnings` and `//noinspection`

→ **Fix all** applies to the whole file in one step

→ Code › **Inspect Code** → results in the Problems window

WHERE IN SETTINGS

⚙ `Editor › Inspections` — profile, severities (Project / IDE)

⚙ `Editor › Intentions` — enable / disable intentions

⚙ `Tools › Actions on Save › Run code cleanup` — apply safe fixes

IN VS CODE

⇄ Quick Fix ⌘. ↔ `⌥⏎`; Problems ⇧⌘M ↔ ⌘6

⇄ Linters (ESLint, SonarLint) give similar warnings · See VS Code 12–13

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Show Context Actions	⌥⏎	Alt+Enter
Next / prev problem	F2 / ⇧F2	F2 / Shift+F2
Problems tool window	⌘6	Alt+6
Error description	⌘F1	Ctrl+F1
Run inspection by name	⌥⇧⌘I	Ctrl+Alt+Shift+I

SEVERITIES

LEVEL	SHOWN AS
Error	red
Warning	yellow
Weak warning	underline
Typo	green squiggle

⚠ COMMON MISTAKES

- Suppressing with `@SuppressWarnings("all")` — hides real bugs; suppress the specific inspection
- Changing severities in an IDE profile — teammates don't get it; use a project profile
- Ignoring red code that compiles — often a stale index; File › Invalidate Caches

✓ TIPS & BEST PRACTICES

- Run **Inspect Code** on a module before a big refactor
- "Run inspection by name" fixes one issue type across the project
- Turn on Java language-level migration inspections when upgrading JDKs

QUICK SUMMARY

- `⌥⏎` — fixes and intentions
- `inspection profile` — share with project
- `F2` — next problem
- `Inspect Code` — batch analysis

Sheet 15

Rename & Extract Refactorings

Rename ⇧F6 · Extract Variable ⌥⌘V · Method ⌥⌘M · Constant ⌥⌘C · Field · Parameter · Refactor This ^T

DESCRIPTION

Refactorings change structure without changing behavior, and IntelliJ updates every reference. **Rename** works on any symbol (classes rename their file). **Extract** pulls an expression or block into a variable, constant, field, parameter or new method; the method extraction finds duplicates and offers to replace them.

NOTES

- Most refactorings run **in place**; press ⇧F6 / ⌥⌘V again for the dialog
- Select an expression first, or the IDE offers candidates
- Rename also updates comments and strings if you ask

BEFORE → AFTER EXTRACT

```
// Before: select the expression, press ⌥⌘V
double total = order.getQty() * order.getPrice() * 1.08;

// Extract Variable (⌥⌘V)
double subtotal = order.getQty() * order.getPrice();
double total = subtotal * 1.08;

// Extract Constant (⌥⌘C) on 1.08
private static final double TAX_RATE = 1.08;
double total = subtotal * TAX_RATE;

// Extract Method (⌥⌘M) on the two lines
double total = totalWithTax(order);
private double totalWithTax(Order order) {
    double subtotal = order.getQty() * order.getPrice();
    return subtotal * TAX_RATE;
}
```

CODE EXPLAINED

- → ⌥⌘V with several matches → "Replace all 3 occurrences?"
- → Extract Method detects parameters and the return value
- → ^T Refactor This — menu of refactorings valid here
- → Rename a class: file, usages, tests and Test classes optional

WHERE IN SETTINGS

- ⚙ Editor › Code Editing › Refactorings — in place vs dialog
- ⚙ Editor › Code Style › Java › Code Generation — naming prefixes
- ⚙ Advanced Settings — rename in comments by default

IN VS CODE

- ⇄ Rename Symbol F2 ↔ ⇧F6 ; Extract via Quick Fix ⌘.
- ⇄ VS Code Java offers fewer extracts (no constant/parameter object) · See VS Code 13

REFACTORING SHORTCUTS

ACTION	MACOS	WIN / LINUX
Rename	⇧F6	Shift+F6
Extract Variable	⌥⌘V	Ctrl+Alt+V
Extract Constant	⌥⌘C	Ctrl+Alt+C
Extract Field	⌥⌘F	Ctrl+Alt+F
Extract Parameter	⌥⌘P	Ctrl+Alt+P
Extract Method	⌥⌘M	Ctrl+Alt+M
Refactor This	^T	Ctrl+Alt+Shift+T

⚠ COMMON MISTAKES

- Find-and-replace to rename — misses overloads and catches unrelated text
- Extracting a method mid-expression with side effects — check the preview; order may change
- Renaming a public API class used by other repos — IntelliJ can't see those callers

✓ TIPS & BEST PRACTICES

- Use Extract Method to split long methods, then rename the pieces
- Undo (⌘Z) reverts a whole refactoring in one step
- Commit before a large rename so the diff is clean

QUICK SUMMARY

- ⇧F6 — rename everywhere
- ⌥⌘V / ⌥⌘C — variable / constant
- ⌥⌘M — extract method
- ^T — refactoring menu

Sheet 16

Change Signature, Move & Inline

Change Signature ⌘F6 · Move F6 · Inline ⌥⌘N · Safe Delete ⌘⌫ · Pull Members Up · parameter object

DESCRIPTION

Change Signature adds, removes or reorders parameters and updates every call site, with a default value for new parameters. **Move** relocates classes and static members, fixing imports. **Inline** does the reverse of extract. **Safe Delete** removes a symbol only after checking it's unused.

NOTES

- Refactorings show a **preview** when usages are ambiguous
- Pull Members Up / Push Members Down work across a hierarchy
- Introduce Parameter Object groups parameters into a record

CHANGE SIGNATURE EXAMPLE

```
// Before
public Invoice bill(Customer c, List<Item> items) { ... }
bill(customer, cart.items());

// ⌘F6 → add parameter: Currency currency, default value: Currency.USD
public Invoice bill(Customer c, List<Item> items, Currency cur) {…}
bill(customer, cart.items(), Currency.USD);     // every call updated

// Introduce Parameter Object (via ^T) → record BillRequest
public Invoice bill(BillRequest request) { ... }
bill(new BillRequest(customer, cart.items(), Currency.USD));

// Inline (⌥⌘N) on a one-line helper removes it and pastes the body
```

CODE EXPLAINED

- → Default value fills existing calls; use **Use any var** to pick a local
- → Delegate via overloading keeps the old signature as a wrapper
- → Move (`F6`) on a static method → pick the target class
- → Safe Delete warns and lists usages if any remain

WHERE IN SETTINGS

- ⚙ `Editor › Code Editing › Refactorings` — show preview
- ⚙ `Editor › Code Style › Java › Imports` — how moved code imports
- ⚙ `Advanced Settings` — Safe Delete search in comments

IN VS CODE

- ⇄ No Change Signature in VS Code core; Java ext has a basic one
- ⇄ Move class via drag in Explorer updates package · See VS Code 13

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Change Signature	⌘F6	Ctrl+F6
Move	F6	F6
Copy class	F5	F5
Inline	⌥⌘N	Ctrl+Alt+N
Safe Delete	⌘⌫ (in Project)	Alt+Delete

HIERARCHY REFACTORINGS

REFACTORING	DOES
Pull Members Up	move to superclass/interface
Push Members Down	move to subclasses
Extract Interface	new interface from methods
Extract Superclass	new parent class
Use Interface Where Possible	widen types

⚠ COMMON MISTAKES

- Deleting with `⌘⌫` in the editor — that deletes a line; Safe Delete is `⌘⌫` in the Project view or via `^T`
- Adding a parameter with no default — every call breaks until fixed by hand
- Moving classes across modules without checking dependencies — creates cycles

✓ TIPS & BEST PRACTICES

- Use delegation ("Keep method as delegate") for public APIs
- Preview → **Exclude** usages you don't want changed
- Introduce Parameter Object once a method has 4+ params

QUICK SUMMARY

- `⌘F6` — change signature
- `F6` — move
- `⌥⌘N` — inline
- `Safe Delete` — remove if unused

Structural Search & Migrations

DESCRIPTION

Structural Search and Replace (SSR) matches code by shape, not text: `$list$.size() == 0` finds every such comparison whatever the variable names. **Migrations** apply library renames in bulk, including **javax → jakarta** (Jakarta EE 02). Inspections with **Fix all** handle Java language-level upgrades.

NOTES

- SSR templates can become custom inspections
- `$name$` variables can have type and count filters
- Refactor › Migrate Packages and Classes lists built-in maps

STRUCTURAL SEARCH & REPLACE TEMPLATES

```
// Search template                          Replace template
$list$.size() == 0                    →    $list$.isEmpty()
new $T$($args$).toString()            →    String.valueOf(new $T$($args$))
if ($c$) { return true; } else { return false; }  →  return $c$;
Collections.unmodifiableList(new ArrayList<>($x$))  →  List.copyOf($x$)

// Filter on $list$: type = java.util.List  (avoids false matches)

// Migrate: Refactor › Migrate Packages and Classes ›
//   "Java EE to Jakarta EE"   javax.persistence.* → jakarta.persistence.*
//   "JUnit 4 → 5" also exists via inspections (Fix all)
```

CODE EXPLAINED

- → Variables match any expression unless filtered
- → Add a type filter (`java.util.List`) to avoid lookalikes
- → **Save template** and enable it as an inspection with a fix
- → Migration preview lists every file before applying

WHERE IN SETTINGS

- ⚙ `Editor › Inspections › Structural search` — your SSR inspections
- ⚙ `Project Structure › Project › Language level` — after upgrading
- ⚙ `Editor › Inspections › Java › Language level migration aids`

IN VS CODE

- ⇄ No structural search in VS Code; regex search only
- ⇄ Migrations: OpenRewrite CLI works in either editor · See VS Code 09

WHERE TO FIND

ACTION	PATH
`Search Structurally`	Edit › Find › Search Structurally
`Replace Structurally`	Edit › Find › Replace Structurally
`Migrate`	Refactor › Migrate Packages and Classes
`Language upgrade fixes`	Code › Inspect Code › Fix all

LANGUAGE-LEVEL FIXES

INSPECTION	TURNS INTO
`anonymous class → lambda`	Java 8
`explicit type → var`	Java 10
`switch → enhanced switch`	Java 14
`instanceof + cast → pattern`	Java 16
`data class → record`	Java 16

⚠ COMMON MISTAKES

- SSR without type filters — matches unrelated code with the same shape
- Running the javax → jakarta migration on Java SE packages — keep `javax.sql`, `javax.crypto`
- Applying "Fix all" across the project in one commit with other changes — impossible to review

✓ TIPS & BEST PRACTICES

- Start SSR from an existing template (Templates list) and edit
- Commit each migration step separately
- Pair with OpenRewrite for build-file changes IntelliJ doesn't touch

QUICK SUMMARY

- `SSR` — search by code shape
- `$var$` — template variables
- `Migrate` — bulk package renames
- `Fix all` — language upgrades

Run Configurations

Run ^R · Debug ^D · Run Anything ^^ · program args · VM options · env vars · before-launch · Services

DESCRIPTION

A **run configuration** describes how to start something: main class, program arguments, VM options, environment, working directory and **before-launch** tasks. IntelliJ creates one when you click a gutter ▶ icon. Save it as a **project file** to share it with the team. The **Services** tool window manages many running apps at once.

NOTES

- `^⇧R` / `Ctrl+Shift+F10` runs the config for the code at the caret
- Modify options (⌥M) reveals hidden fields like env files
- Templates set defaults for every new config of a type

SHARED CONFIG: .RUN/SHOP.RUN.XML

```
<component name="ProjectRunConfigurationManager">
  <configuration name="Shop" type="Application">
    <option name="MAIN_CLASS_NAME" value="com.shop.ShopApp"/>
    <module name="shop"/>
    <option name="PROGRAM_PARAMETERS" value="--port 8080"/>
    <option name="VM_PARAMETERS" value="-Xmx512m"/>
    <envs><env name="DB_URL" value="jdbc:postgresql:shop"/></envs>
    <method v="2"><option name="Make" enabled="true"/></method>
  </configuration>
</component>
```

CODE EXPLAINED

→ **Store as project file** writes `.run/*.run.xml` — commit it

→ `PROGRAM_PARAMETERS` → `args[]`; `VM_PARAMETERS` → JVM flags

→ `Make` before launch compiles first; add Maven goals or other configs too

→ Allow multiple instances: Modify options › Allow multiple

WHERE IN SETTINGS

⚙ `Run › Edit Configurations › Edit configuration templates` — defaults

⚙ `Build… › Build Tools › Maven/Gradle › Runner` — delegate run to build tool

⚙ `Advanced Settings › Run/Debug` — confirm rerun, console limits

IN VS CODE

⇄ `launch.json` ↔ run configurations; `preLaunchTask` ↔ before-launch

⇄ Run and Debug view ↔ run widget · See VS Code 17

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Run	^R	Shift+F10
Debug	^D	Shift+F9
Run context config	^⇧R	Ctrl+Shift+F10
Run Anything	^^	Ctrl Ctrl
Stop	⌘F2	Ctrl+F2
Services tool window	⌘8	Alt+8

COMMON CONFIG TYPES

TYPE	RUNS
Application	a `main` class
JUnit	tests
Maven / Gradle	build goals / tasks
Spring Boot	Boot app (subscription)
Compound	several configs at once
Remote JVM Debug	attach debugger

⚠ COMMON MISTAKES

- Hard-coding secrets in shared run configs — they end up in Git; use env files ignored by Git
- Changing a temporary config and losing it — click **Save Configuration** to keep it
- Duplicating configs per developer — share one and parameterize with env vars

✓ TIPS & BEST PRACTICES

- Run Anything (`^^`) runs `mvn …`, `gradle …` and configs by name
- Use Compound configs to start backend + frontend together
- Add an `.env` file via Modify options › Environment variables › EnvFile

QUICK SUMMARY

- `run config` — how to start
- `^R / ^D` — run / debug
- `.run/*.run.xml` — shared configs
- `^^` — Run Anything

Sheet 19

Debugger Basics

breakpoints ⌘F8 · Step Over F8 · Into F7 · Out ⇧F8 · Run to Cursor ⌥F9 · Variables · Watches

DESCRIPTION

Start any run configuration with **Debug** (^D) and execution stops at **breakpoints**. Step through line by line, inspect **Variables**, add **Watches**, and move through **Frames** (the call stack). Inline values show each variable's value right in the editor.

NOTES

- Click the gutter to toggle a line breakpoint
- Smart Step Into (`⇧F7`) picks which call on a line to enter
- Debug tool window: `⌘5` / `Alt+5`
- New in 2026.2: logpoints from the gutter menu

DEBUGGER KEYS

ACTION	MACOS	WIN / LINUX
Toggle breakpoint	`⌘F8`	`Ctrl+F8`
View breakpoints	`⇧⌘F8`	`Ctrl+Shift+F8`
Step Over	`F8`	`F8`
Step Into	`F7`	`F7`
Smart Step Into	`⇧F7`	`Shift+F7`
Step Out	`⇧F8`	`Shift+F8`
Run to Cursor	`⌥F9`	`Alt+F9`
Resume	`⌥⌘R`	`F9`
Evaluate Expression	`⌥F8`	`Alt+F8`
Stop	`⌘F2`	`Ctrl+F2`
Mute breakpoints	`Debug toolbar`	`same`

EXPLAINED

→ Variables pane: right-click › **Set Value** to change state live

→ Watches: `+` then any expression; evaluated at each stop

→ Frames: click a caller frame to inspect its locals

→ **Mute breakpoints** runs to the end without stopping

WHERE IN SETTINGS

⚙ `Build… › Debugger` — stepping filters, HotSwap

⚙ `Build… › Debugger › Data Views › Java` — toString and renderers

⚙ `Build… › Debugger › Stepping` — skip getters, synthetic methods

IN VS CODE

⇄ F9 toggles breakpoint in VS Code ↔ `⌘F8` ; F10/F11 step ↔ F8/F7

⇄ Debug Console ↔ Evaluate Expression · See VS Code 16

PANES

PANE	SHOWS
`Frames`	call stack per thread
`Variables`	locals, `this`, fields
`Watches`	your expressions
`Console`	program output
`Threads & Variables`	combined view (new UI)

BREAKPOINT ICONS

ICON	MEANING
`red dot`	line breakpoint
`red diamond`	method breakpoint (slow)
`red lightning`	exception breakpoint
`grey dot`	disabled / muted

⚠ COMMON MISTAKES

- Method breakpoints on hot code — they slow the JVM dramatically; use line breakpoints
- Stepping into JDK internals — enable stepping filters or use Smart Step Into
- Evaluating expressions with side effects (`list.remove(0)`) — they really run

✓ TIPS & BEST PRACTICES

- Use Run to Cursor instead of temporary breakpoints
- Pin values in the editor to keep them visible across steps
- Right-click a frame › Drop Frame to rerun a method (sheet 20)

QUICK SUMMARY

- `⌘F8` — breakpoint
- `F8 / F7 / ⇧F8` — over / into / out
- `⌥F9` — run to cursor
- `⌥F8` — evaluate

Advanced Debugging

DESCRIPTION

Breakpoints can do much more than stop. A **condition** stops only when an expression is true; a **logpoint** prints and continues; an **exception breakpoint** stops where an exception is thrown. **HotSwap** reloads changed method bodies without restarting, and **remote debug** attaches to any JVM started with the JDWP agent.

NOTES

- Right-click a breakpoint for condition, log and suspend options
- Exception breakpoints: `⇧⌘F8` › + › Java Exception Breakpoints
- Stream Debugger visualizes each step of a stream chain

REMOTE DEBUG & BREAKPOINT SETTINGS

```
# Start the JVM with the debug agent (Java 9+)
java -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005 \
     -jar shop.jar

# Docker: expose the port too
docker run -p 5005:5005 -e JAVA_TOOL_OPTIONS=\
"-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005" shop

# Run › Edit Configurations › + › Remote JVM Debug → host:5005

# Breakpoint options (right-click the red dot):
#   Condition:     order.total() > 1000
#   Log:           "order " + order.id()  + uncheck Suspend → logpoint
#   Pass count:    stop on the 50th hit
#   Instance filter / Caller filter
```

CODE EXPLAINED

- → `suspend=y` waits for the debugger before starting (debug startup code)
- → `address=*:5005` listens on all interfaces — never expose publicly
- → HotSwap: Build (`⌘F9`) while paused → "Reload changed classes"
- → Drop Frame rewinds to the start of the selected method

WHERE IN SETTINGS

- ⚙ `Build… › Debugger › HotSwap` — reload after compilation
- ⚙ `Build… › Debugger › Async Stack Traces` — capture points
- ⚙ `Build… › Debugger › Data Views` — custom renderers

IN VS CODE

- ⇄ Logpoints and conditional breakpoints exist in VS Code too
- ⇄ Remote attach via `"request": "attach"` in `launch.json` · See VS Code 16–17

BREAKPOINT TYPES

TYPE	STOPS WHEN
`line + condition`	expression is true
`logpoint`	never; logs a message
`exception`	exception thrown (caught/uncaught)
`field watchpoint`	field read / written
`method`	method entry/exit (slow)

HOTSWAP LIMITS

CHANGE	RELOADS?
`method body`	`yes`
`add method / field`	`no (standard JVM)`
`change signature`	`no`
`JBR / DCEVM`	more changes allowed

⚠ COMMON MISTAKES

- Leaving `suspend=y` in a container — the app hangs until a debugger attaches
- Conditions that call slow methods — evaluated on every hit; the app crawls
- Expecting HotSwap to apply new fields — restart the app

✓ TIPS & BEST PRACTICES

- Use logpoints instead of adding `println` and rebuilding
- Exception breakpoint on `NullPointerException` with "caught" off finds the real origin
- Mark objects (F11 on a value) to track the same instance across frames

QUICK SUMMARY

- `condition` — stop selectively
- `logpoint` — log without stopping
- `exception breakpoint` — stop at throw
- `-agentlib:jdwp` — remote debug

Testing & Coverage

DESCRIPTION

IntelliJ runs JUnit, TestNG and Spock tests from the **gutter**, a class, a package or a whole module. **Go to Test** jumps between a class and its test, and creates one if it doesn't exist. The test runner shows a tree of results; **Run with Coverage** highlights covered lines in the editor. The JUnit set covers JUnit itself.

NOTES

- Green/red gutter icons show last result per test
- Tests can delegate to Maven/Gradle (`Build Tools › Gradle › Run tests using`)
- Testcontainers: gutter shows container status; Docker plugin manages images

PLUGIN PICK

+ **JUnit 5 Mockito Code Generator** — generates test skeletons with `@Mock` / `@InjectMocks` fields

TEST SHORTCUTS

ACTION	MACOS	WIN / LINUX
Go to / Create Test	⇧⌘T	Ctrl+Shift+T
Run test at caret	^⇧R	Ctrl+Shift+F10
Debug test at caret	^⇧D	Ctrl+Shift+F9 (via ⌥⏎)
Rerun last	^R	Shift+F10
Rerun failed tests	Run toolbar ↻✖	same
Run with coverage	Run widget ⋮ › Coverage	same
Toggle auto-test	Run toolbar	same
Next failed test	⌘⌥↓ in runner	Ctrl+Alt+↓

EXPLAINED

→ ⇧⌘T on a class with no test → **Create Test** dialog (library, methods)

→ Runner tree: filter passed, show ignored, sort by duration

→ Coverage gutter: green covered, red not, yellow partial branches

→ **Toggle auto-test** reruns tests when code changes

WHERE IN SETTINGS

⚙ `Build… › Coverage` — runner (IntelliJ / JaCoCo), branch coverage

⚙ `Build… › Build Tools › Gradle` — run tests using Gradle vs IntelliJ

⚙ `Editor › File and Code Templates › Code › JUnit5 Test Class`

IN VS CODE

⇄ Testing view + gutter ▶ in both; Test Runner for Java extension

⇄ Coverage view ↔ Run with Coverage · See VS Code 19

CREATE TEST DIALOG

FIELD	CHOOSE
Testing library	JUnit 5 / 6, TestNG, Spock
Class name	OrderServiceTest
Destination	matching test package
Generate	setUp/@BeforeEach, test methods

COVERAGE RUNNERS

RUNNER	NOTES
IntelliJ IDEA	fast, line + branch
JaCoCo	matches CI reports
exclusions	Edit config › Code Coverage

⚠ COMMON MISTAKES

- Running tests with IntelliJ while CI uses Gradle — different results from build-only config; delegate to Gradle
- Reading 100% line coverage as "tested" — check branch coverage and assertions
- Leaving auto-test on with slow integration tests — CPU stays busy

✓ TIPS & BEST PRACTICES

- Rerun failed tests first, then the whole suite
- Use the runner's **Export Test Results** for sharing
- Mark the test root as **Test Sources Root** so ⇧⌘T finds the right package

QUICK SUMMARY

- ⇧⌘T — go to / create test
- gutter ▶ — run one test
- rerun failed — fast loop
- coverage — gutter highlights

Sheet 22 Subscription

Profiler & JVM Tools

built-in profiler · CPU & allocation flame graphs · attach to JVM · memory snapshots · thread dumps · JFR

DESCRIPTION

The built-in **profiler** samples CPU time and memory allocations using async-profiler and **Java Flight Recorder**, then shows **flame graphs**, call trees and method lists. Start any run configuration with the profiler, or **attach** to a running JVM. **Inline hints** show CPU time beside your code after a run.

NOTES

- Profiler is a subscription feature in the unified IDE
- Capture memory snapshots (`.hprof`) and analyze in the IDE
- Thread dumps: Debug toolbar camera icon, or `jstack`
- Linux/macOS use async-profiler; Windows uses JFR

PLUGIN PICK

+ **VisualVM Launcher** — opens VisualVM for the run; the built-in profiler covers most needs

WHERE TO FIND IT

→ Profile a run → Run widget ⋮ › Profile

→ Attach → Profiler tool window › running JVMs

→ Snapshot files → `File › Open` (`.jfr` , `.hprof`)

→ Inline CPU hints → editor gutter after profiling

STEPS

1. Run widget ⋮ › **Profile with IntelliJ Profiler**
2. Exercise the slow path in the app
3. Stop or click **Capture snapshot**
4. Open **Flame Graph** — wide bars are where time goes
5. Switch to **Call Tree** / **Method List**; double-click to jump to code
6. Change view to **Memory Allocations** to find allocation hotspots

WHERE IN SETTINGS

⚙ `Build… › Java Profiler` — configurations (async / JFR), sampling

⚙ `Build… › Java Profiler` — show inline hints

⚙ `Appearance & Behavior › Path Variables` — snapshot location

IN VS CODE

⇄ No built-in profiler; use VisualVM, JDK Mission Control or async-profiler

⇄ Java extension debugger has no flame graphs · See VS Code 16

VIEWS

VIEW	ANSWERS
`Flame Graph`	where does time/memory go?
`Call Tree`	which caller path?
`Method List`	hottest methods
`Timeline`	what were threads doing?
`Events`	JFR events (GC, locks)

JDK TOOLS

TOOL	USE
`jcmd <pid> JFR.start`	start a recording
`jcmd <pid> Thread.print`	thread dump
`jcmd <pid> GC.heap_dump f.hprof`	heap dump
`jfr print f.jfr`	read a recording

⚠ COMMON MISTAKES

- Profiling a debug run with breakpoints — timings are meaningless
- Profiling for a few milliseconds — too few samples; run the slow path repeatedly
- Optimizing the widest frame without reading its children — the cost may be in a callee

✓ TIPS & BEST PRACTICES

- Compare two snapshots (before / after) with **Compare with**
- Warm up the JVM before capturing
- Use the Timeline to spot lock contention and GC pauses

QUICK SUMMARY

- `Profile` — run with profiler
- `Flame Graph` — hotspots
- `Attach` — running JVM
- `JFR` — low-overhead recording

Maven in IntelliJ

DESCRIPTION

IntelliJ imports the project model from `pom.xml`. After editing the pom, **reload** (⇧⌘I) to sync. The **Maven tool window** runs lifecycle phases and plugin goals, toggles **profiles**, and the **Dependency Analyzer** shows conflicts. The Maven set covers Maven itself.

NOTES

- The floating 🐘 /M icon appears when the pom changes — click to reload
- Uses the Maven wrapper (`mvnw`) if present
- Double-click a goal to run; right-click to create a run config

POM.XML EDITING HELPERS

```
<dependencies>
  <!-- ⌘N in <dependencies> › Add Maven Artifact Dependency -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <!-- ^Space in <version> lists available versions -->
    <version>5.13.4</version>
    <scope>test</scope>
  </dependency>
</dependencies>
<!-- ⌘B on a dependency opens its pom; ⌥⏎ › update version -->
<!-- Right-click pom › Maven › Show Dependencies (diagram) -->
```

CODE EXPLAINED

- → `⌘N` inside the pom searches Maven Central
- → Version completion and update intentions keep versions current
- → **Show Dependencies** draws the graph; red lines are conflicts
- → Run Anything (`^^`) + `mvn clean verify` runs any command

WHERE IN SETTINGS

- ⚙ `Build… › Build Tools › Maven` — Maven home (wrapper/bundled), settings.xml
- ⚙ `Build… › Build Tools › Maven › Importing` — JDK for importer
- ⚙ `Build… › Build Tools` — auto-reload after changes

IN VS CODE

- ⇄ Maven for Java extension adds a Maven view
- ⇄ Reload via "Java: Reload Projects" · See VS Code 23

MAVEN TOOL WINDOW

ITEM	DOES
⟳ Reload All	re-import every module
Lifecycle	clean, compile, test, package…
Plugins	every goal
Profiles	checkboxes = –P
⚡ Offline	–o
Skip tests	–DskipTests toggle

SHORTCUTS

ACTION	MACOS	WIN / LINUX
Reload Maven changes	⇧⌘I	Ctrl+Shift+O
Run Anything	^^	Ctrl Ctrl
Dependency Analyzer	tool window	same

⚠ COMMON MISTAKES

- Editing Project Structure modules instead of the pom — overwritten on reload
- Forgetting to reload after pom edits — red imports that disappear after sync
- Profiles ticked in IntelliJ but not in CI — builds differ

✓ TIPS & BEST PRACTICES

- Use the Dependency Analyzer's **Show Conflicts** before bumping versions
- Delegate build/run to Maven if the build has custom plugins
- Keep `mvnw` committed so IDE and CI use the same Maven

QUICK SUMMARY

- `⇧⌘I` — reload pom
- `Maven tool window` — goals & profiles
- `Dependency Analyzer` — conflicts
- `⌘N in pom` — add dependency

Gradle in IntelliJ

DESCRIPTION

IntelliJ **syncs** its model from `build.gradle(.kts)`. By default it **builds and runs using Gradle**, so results match the command line; switching to IntelliJ's own builder is faster for some projects. The **Gradle tool window** lists tasks, dependencies and source sets. Kotlin DSL gets full completion and navigation.

NOTES

- Sync: `⇧⌘I` / `Ctrl+Shift+O` or the floating icon
- **Gradle JVM** is separate from the project SDK
- 2026.2 adds early Gradle 10 support and migration help

BUILD.GRADLE.KTS HELPERS

```
plugins {
    java
    id("org.springframework.boot") version "4.1.0"   // ^Space for versions
}
dependencies {
    // ⌘N › Add dependency (searches Maven Central)
    implementation("com.fasterxml.jackson.core:jackson-databind")
    testImplementation(platform("org.junit:junit-bom:5.13.4"))
    testImplementation("org.junit.jupiter:junit-jupiter")
}
tasks.test { useJUnitPlatform() }    // ⌘B on useJUnitPlatform → source
// Gradle tool window › Tasks › verification › test (double-click)
```

CODE EXPLAINED

- → Dependency completion arrived in 2026.2 for Gradle scripts
- → `⌘B` navigates into Gradle API and plugin sources
- → Double-click a task to run; it becomes a run config
- → Version catalogs (`libs.versions.toml`) get completion and navigation

WHERE IN SETTINGS

- ⚙ `Build… › Build Tools › Gradle` — Gradle JVM, build/run/test using
- ⚙ `Build… › Build Tools › Gradle` — distribution (wrapper vs local)
- ⚙ `Build… › Build Tools` — reload after changes

IN VS CODE

- ⇄ Gradle for Java extension adds a Gradle view
- ⇄ Same wrapper; sync via "Java: Reload Projects" · See VS Code 23

BUILD & RUN USING

OPTION	TRADE-OFF
`Gradle (default)`	matches CLI and CI
`IntelliJ IDEA`	faster incremental, may differ
`Run tests using`	set separately

SHORTCUTS

ACTION	MACOS	WIN / LINUX
`Sync Gradle changes`	`⇧⌘I`	Ctrl+Shift+O
`Run task`	`^^ gradle test`	Ctrl Ctrl
`Execute task dialog`	`Gradle window` ⌨	same

⚠ COMMON MISTAKES

- Gradle JVM older than the build requires — sync fails with unsupported class version
- Switching build to IntelliJ without matching annotation processors — compile errors only in the IDE
- Not committing the wrapper — IDE downloads a different Gradle than CI

✓ TIPS & BEST PRACTICES

- Enable Gradle configuration cache for faster syncs
- Use **Analyze Dependencies** in the tool window for conflicts
- Keep `gradle.properties` org.gradle.jvmargs sized for big projects

QUICK SUMMARY

- `⇧⌘I` — sync
- `Gradle JVM` — runs Gradle itself
- `build using Gradle` — CLI parity
- `Kotlin DSL` — full IDE support

Sheet 25 Subscription

Spring & Jakarta EE Support

Spring bean navigation · Endpoints tool window · `application.properties` completion · CDI checks · app servers

DESCRIPTION

With a subscription, IntelliJ understands **Spring** and **Jakarta EE** models: gutter icons jump between beans and injection points, `application.yml` keys complete and validate, and the **Endpoints** tool window lists every HTTP route. Jakarta support checks **CDI** injection, Persistence mappings and lets you run apps on **WildFly, Tomcat, Payara** and others.

NOTES

- 2026.2 adds richer Spring Security insights and DB migration workflows
- Spring Boot run configs show actuator beans and health
- See the Spring and Jakarta EE sets for the frameworks

WHAT THE IDE CHECKS

```
@Service                                  // gutter     → bean definition
public class OrderService {
    private final PaymentGateway gateway;   // gutter → which bean injects
    public OrderService(PaymentGateway g) { this.gateway = g; }
}
// ⚠ "Could not autowire. There is more than one bean of type ..."

@ApplicationScoped                        // Jakarta CDI
public class Checkout {
    @Inject Shipping shipping;            // ⚠ ambiguous dependency
}

@RestController @RequestMapping("/api/orders")
public class OrderController {
    @GetMapping("/{id}")                  // gutter 🌐 → HTTP Client
    Order one(@PathVariable long id) { ... }
}
```

CODE EXPLAINED

- → Bean gutter icons: navigate to definitions and injection points
- → Ambiguous / unsatisfied dependencies are flagged before you run
- → Endpoint gutter icon generates an HTTP Client request (sheet 26)
- → Spring Boot config keys: completion, docs, unused-key warnings

WHERE IN SETTINGS

- ⚙ `Languages & Frameworks › Spring` — contexts, Boot config files
- ⚙ `Build… › Application Servers` — register WildFly / Tomcat
- ⚙ `Languages & Frameworks › JPA` — default provider, console

IN VS CODE

- ⇄ Spring Boot Extension Pack / Jakarta extensions for VS Code
- ⇄ Fewer cross-bean checks; no app-server integration · See VS Code 23

TOOL WINDOWS

WINDOW	SHOWS
Endpoints	all HTTP / WebSocket routes
Spring › Beans	bean graph per context
Persistence	entities, units, JPQL console
Services	running Boot apps / servers

APP SERVER RUN CONFIGS

SERVER	CONFIG TYPE
Tomcat	Tomcat Server › Local
WildFly	JBoss/WildFly Server
Payara / GlassFish	GlassFish Server
Open Liberty	Liberty plugin

⚠ COMMON MISTAKES

- Expecting Spring inspections on the free tier — they need a subscription
- Ignoring "could not autowire" in tests with component scan differences — check the test context

✓ TIPS & BEST PRACTICES

- Use Endpoints to discover an unfamiliar API quickly
- Turn on Boot config metadata generation for your own properties
- Open Liberty's Maven dev mode works well alongside the IDE

QUICK SUMMARY

- `bean gutter` — navigate DI
- `Endpoints` — all routes
- `config completion` — Boot keys
- `app servers` — Jakarta deploy

Sheet 26 Subscription

HTTP Client & Database Tools

.http files · ### requests · http-client.env.json · response handlers · Database tool window · SQL console

DESCRIPTION

The **HTTP Client** runs requests stored in `.http` files: commit them next to the code as runnable API docs. **Environments** swap hosts and tokens; **response handlers** (JavaScript) save values for later requests. The **Database** tool window connects to data sources, browses schemas and runs SQL in **consoles** with completion against the real schema.

NOTES

- Basic database tools are free; HTTP Client and full DB tools need a subscription
- `http-client.private.env.json` for secrets (git-ignored)
- Run a request with the gutter ▶
- SQL console: `⌘⏎` / `Ctrl+Enter` executes

API.HTTP

```
### List products
GET {{host}}/api/products?size=20
Accept: application/json

### Log in and keep the token
POST {{host}}/api/login
Content-Type: application/json

{ "email": "bob@shop.app", "password": "{{password}}" }

> {% client.global.set("token", response.body.token); %}

### Create a product with the saved token
POST {{host}}/api/products
Authorization: Bearer {{token}}
Content-Type: application/json

{ "sku": "MUG-1", "price": 9.50 }
```

CODE EXPLAINED

- → `###` separates requests; text after it is the name
- → `{{host}}` — from the selected environment file
- → `> {% … %}` — JS handler; `client.global.set` stores variables
- → Responses are saved in `.idea/httpRequests/` history

WHERE IN SETTINGS

- ⚙ `Tools › HTTP Client` — proxy, SSL, history size
- ⚙ `Database › Data Sources` (tool window) — drivers, SSH tunnels
- ⚙ `Editor › Code Style › SQL` — SQL formatting

IN VS CODE

- ⇄ REST Client extension uses the same `.http` format (mostly)
- ⇄ SQLTools / Database Client extensions ≈ Database tool window · See VS Code 23

HTTP-CLIENT.ENV.JSON

KEY	EXAMPLE
`dev.host`	`http://localhost:8080`
`prod.host`	`https://api.shop.app`
`private: password`	in `.private.env.json`

DATABASE SHORTCUTS

ACTION	MACOS	WIN / LINUX
Execute statement	⌘⏎	Ctrl+Enter
New console	⇧⌘L	Ctrl+Shift+L
Open table data	F4	F4
Explain plan	context menu	same

⚠ COMMON MISTAKES

- Committing tokens in `http-client.env.json` — use the private env file
- Running destructive SQL on the wrong data source — color-code prod connections
- Forgetting to select an environment — `{{host}}` stays unresolved

✓ TIPS & BEST PRACTICES

- Generate requests from the Endpoints window or a controller gutter
- Convert cURL: paste a `curl` command into a `.http` file
- Set prod data sources to **read-only** in their properties

QUICK SUMMARY

- `.http` — runnable requests
- `env.json` — per-environment vars
- `> {% %}` — response handler
- `Database` — consoles & schema

Git in IntelliJ

DESCRIPTION

Git is built in. The **Commit** tool window (⌘0) shows changes; tick files or individual **chunks** for a partial commit. **Changelists** group local changes by task. The **branch widget** in the toolbar handles checkout, new branches, merges and rebases. **Shelve** is IntelliJ's own stash; **stash** is Git's.

NOTES

- Gutter color bars: green added, blue modified, grey deleted
- Click a gutter bar to see, revert or stage that hunk
- Commit checks: reformat, inspections, tests before commit

PLUGIN PICK

+ **GitToolBox** (zielu) — inline blame, ahead/behind counts, auto-fetch

GIT SHORTCUTS

ACTION	MACOS	WIN / LINUX
Commit	⌘K	Ctrl+K
Push	⇧⌘K	Ctrl+Shift+K
Update Project (pull)	⌘T	Ctrl+T
VCS Operations popup	^V	Alt+`
Commit tool window	⌘0	Alt+0
Git tool window (log)	⌘9	Alt+9
Rollback changes	⌥⌘Z	Ctrl+Alt+Z
Show diff	⌘D	Ctrl+D
Branches	toolbar branch widget	same

EXPLAINED

→ Partial commit: open the diff, tick only the lines you want

→ **Amend** checkbox in the commit window rewrites the last commit

→ Shelve keeps patches in `.idea/shelf`; stash stays in Git

→ Update Project: merge or rebase, chosen per pull

WHERE IN SETTINGS

⚙ `Version Control › Git` — path to git, update method, auto-fetch

⚙ `Version Control › Commit` — commit checks, non-modal window

⚙ `Version Control › Changelists` — create per task

IN VS CODE

⇄ Source Control ^⇧G ↔ Commit ⌘0; stage lines in both

⇄ GitLens ↔ GitToolBox · See VS Code 20

SHELVE VS STASH

ASPECT	SHELVE	STASH
stored in	IDE (`.idea/shelf`)	Git repo
partial	files / chunks	whole working tree
visible to CLI	no	yes

BRANCH WIDGET ACTIONS

ACTION	DOES
New Branch	create + checkout
Checkout and Rebase onto Current	rebase another branch
Merge into Current	merge
Compare with Current	diff branches

⚠ COMMON MISTAKES

- Committing unrelated changes together — use changelists or partial commits
- Shelving and expecting a teammate to see it — shelves are local
- Force-pushing from the push dialog without checking — use `--force-with-lease` (enabled by default)

✓ TIPS & BEST PRACTICES

- Enable "Reformat code" and "Optimize imports" in commit checks
- Use the VCS Operations popup (`^V`) as a Git command menu
- The Git log (`⌘9`) filters by branch, user, path and date

QUICK SUMMARY

- `⌘K` — commit
- `⇧⌘K` — push
- `⌘T` — update
- `changelists` — group changes

Sheet 28

Diff, Merge & History

diff viewer ⌘D · three-way merge · resolve conflicts · Annotate with Git Blame · history · rebase · cherry-pick

DESCRIPTION

The **diff viewer** compares any two versions side by side or unified. On conflicts, the **three-way merge** tool shows yours, theirs and the result, with one-click accept per chunk and a magic wand for non-conflicting changes. **Annotate** (blame) shows who changed each line; history works per file or per selection.

NOTES

- 2026.2 streamlines conflict resolution into one flow
- Interactive rebase: Git log › right-click › Interactively Rebase from Here
- Cherry-pick: right-click a commit in the log

STEPS

1. When a merge stops, open **Resolve Conflicts** from the notification
2. Pick a file › **Merge…** to open the three-way view
3. Click ✨ (Apply non-conflicting changes) first
4. For each conflict, accept left `>>`, right `<<`, or edit the middle
5. **Apply**, then commit the merge
6. Use Annotate (right-click gutter › Annotate with Git Blame) to see line authors

WHERE TO FIND IT

- → Resolve → `Git › Resolve Conflicts`
- → Blame → gutter right-click › Annotate with Git Blame
- → File history → right-click › Git › Show History
- → Selection history → select › Git › Show History for Selection

WHERE IN SETTINGS

- ⚙ `Version Control › Git` — rebase vs merge for Update Project
- ⚙ `Tools › Diff & Merge` — ignore whitespace, external tools
- ⚙ `Version Control › Confirmation` — force push protection

IN VS CODE

- ⇄ 3-way merge editor exists in VS Code too
- ⇄ Blame via GitLens; `code -d a b` ↔ `idea diff a b` · See VS Code 21

DIFF SHORTCUTS

ACTION	MACOS	WIN / LINUX
`Show diff`	`⌘D`	Ctrl+D
`Next difference`	`F7`	F7
`Previous difference`	`⇧F7`	Shift+F7
`Compare with clipboard`	`Find Action`	same

INTERACTIVE REBASE ACTIONS

ACTION	EFFECT
`Pick`	keep commit
`Reword`	edit message
`Squash / Fixup`	merge into previous
`Drop`	remove commit
`Edit`	stop to amend

⚠ COMMON MISTAKES

- Accepting "theirs" for a whole file to finish fast — silently drops your changes
- Rebasing shared branches from the log — rewrites history others pulled
- Ignoring whitespace in diffs, then committing whitespace-only churn

✓ TIPS & BEST PRACTICES

- Compare branches with Git › Compare with Branch before merging
- Annotate › click a line to open its commit
- Use Show History for Selection to trace one method

QUICK SUMMARY

- `⌘D` — diff
- `three-way merge` — resolve conflicts
- `Annotate` — blame
- `interactive rebase` — edit history

AI Assistant & Agents

AI chat · inline prompts · AI completion · Junie agent · third-party agents · MCP servers · commit messages

DESCRIPTION

IntelliJ's AI features come in layers: free local **full-line completion**, cloud **AI completion** and **AI chat**, and **agents** that make multi-file changes (JetBrains' **Junie**, plus third-party agents such as Claude Code and Copilot). 2026.2 added native **GitHub Copilot** integration and **agent skills**. MCP servers give agents access to tools and data.

NOTES

- Cloud AI needs a JetBrains AI plan (a free tier exists)
- Project rules: `.junie/guidelines.md` and similar files steer agents
- AI features may be disabled by org policy
- The Agentic Coding set covers prompting and workflows

PLUGIN PICK

- **Claude Code** (Anthropic) — runs Claude Code with IDE diff view and selection context
- **GitHub Copilot** — now natively integrated in 2026.2

STEPS

1. Open AI chat from the right tool window stripe
2. Attach context: current file, selection, or `@` mentions
3. Ask for a change; review the diff before **Apply**
4. For multi-file work, switch to an agent (Junie or a third-party agent)
5. Approve tool and terminal actions as the agent requests them
6. Use **Generate Commit Message** in the commit window

WHERE TO FIND IT

→ AI chat → right tool window stripe

→ Inline prompt → editor right-click › AI Actions

→ MCP servers → `Settings › Tools › AI Assistant › MCP`

→ Commit message → ✨ in commit window

WHERE IN SETTINGS

⚙ `Tools › AI Assistant` — models, providers, data sharing

⚙ `Editor › General › Inline Completion` — local / cloud completion

⚙ `Tools › AI Assistant › Model Context Protocol (MCP)` — servers

IN VS CODE

⇄ Copilot Chat / agent mode ↔ AI chat / Junie

⇄ `.vscode/mcp.json` ↔ IDE MCP settings · See VS Code 26–27

AI FEATURES

FEATURE	RUNS
`Full-line completion`	local model, free
`AI completion`	cloud, multi-line
`AI chat`	cloud, ask / edit
`Junie`	agent, multi-step tasks
`Third-party agents`	Claude Code, Copilot, Codex via plugins

GUARDRAILS

SETTING	WHY
`require approval for terminal`	no surprise commands
`exclude files (.aiignore)`	keep secrets out
`review diffs`	agents can be wrong

⚠ COMMON MISTAKES

- Applying AI diffs without running tests — confident-looking code can still be wrong
- Sharing secrets in prompts or context files — exclude them with `.aiignore`
- Assuming feature names stay the same — AI menus change often between releases

✓ TIPS & BEST PRACTICES

- Keep a project guidelines file so agents follow your conventions
- Ask the agent to write tests first, then the change
- Use the local completion when offline or for privacy

QUICK SUMMARY

- `full-line completion` — local, free
- `AI chat` — ask & edit
- `Junie / agents` — multi-file tasks
- `MCP` — tools for agents

Sheet 30 · Subscription

Remote Development & Collaboration

remote development over SSH · Dev Containers · WSL · Code With Me · shared run configurations · settings to commit

DESCRIPTION

Remote Development runs the IDE backend on another machine (SSH server, cloud VM) while a thin client shows the UI locally. **Dev Containers** open a project inside a container defined by `devcontainer.json`. **WSL** projects work natively on Windows. For teamwork, commit shared run configurations, code style and inspection profiles.

NOTES

- Start from **File › Remote Development** or the Welcome screen
- The backend needs a few GB of RAM on the remote machine
- Code With Me's status changes between releases; check before relying on it

.DEVCONTAINER/DEVCONTAINER.JSON

```
{
  "name": "shop",
  "image": "mcr.microsoft.com/devcontainers/java:25",
  "features": {
    "ghcr.io/devcontainers/features/docker-in-docker:2": {}
  },
  "forwardPorts": [8080, 5432],
  "postCreateCommand": "./mvnw -q dependency:go-offline",
  "customizations": {
    "jetbrains": { "backend": "IntelliJ" }
  }
}
```

CODE EXPLAINED

- → Same `devcontainer.json` format as VS Code
- → `customizations.jetbrains.backend` — which IDE backend to install
- → `forwardPorts` — reachable from your laptop
- → `postCreateCommand` — warm caches once

WHERE IN SETTINGS

- ⚙ `Build… › Docker` — Docker connection for Dev Containers
- ⚙ `Tools › SSH Configurations` — saved hosts
- ⚙ `Build… › Deployment` — SFTP mappings (older workflow)

IN VS CODE

- ⇄ Remote - SSH / Dev Containers / WSL extensions ↔ JetBrains Gateway / Remote Development
- ⇄ Live Share ↔ Code With Me · See VS Code 25

COMMIT FOR THE TEAM

PATH	SHARES
`.run/`	run configurations
`.idea/codeStyles/`	code style
`.idea/inspectionProfiles/`	inspections
`.editorconfig`	cross-editor style
`.devcontainer/`	dev environment

REMOTE OPTIONS

OPTION	BEST FOR
`SSH backend`	big servers, cloud VMs
`Dev Container`	reproducible setup
`WSL`	Linux toolchain on Windows

⚠ COMMON MISTAKES

- Running a full IDE over SSH X-forwarding — slow; use the remote backend instead
- Remote backend on a tiny VM — indexing stalls; give it enough RAM and CPU
- Different JDKs in the container and locally — tests pass in one place only

✓ TIPS & BEST PRACTICES

- Keep the dev container image in the repo so onboarding is one click
- Forward database ports to use the local Database tool window
- Share settings through files, not screenshots of Settings

QUICK SUMMARY

- `Remote Development` — backend on server
- `Dev Containers` — project in a container
- `WSL` — Linux on Windows
- `.run/ + .idea/codeStyles` — team files

Sheet 31

Quick Reference: Keyboard Shortcuts

macOS · Windows / Linux · general · editing · navigation · search · refactoring · run & debug · VCS

GENERAL & EDITING (03–08)

ACTION	MACOS	WIN / LINUX
Search Everywhere	⇧⇧	Shift Shift
Find Action	⇧⌘A	Ctrl+Shift+A
Settings	⌘,	Ctrl+Alt+S
Recent Files	⌘E	Ctrl+E
Show Context Actions	⌥⏎	Alt+Enter
Basic / type completion	^Space / ^⇧Space	Ctrl+Space / +Shift
Complete statement	⇧⌘⏎	Ctrl+Shift+Enter
Duplicate line	⌘D	Ctrl+D
Delete line	⌘⌫	Ctrl+Y
Extend selection	⌥↑	Ctrl+W
Next occurrence caret	^G	Alt+J
Comment line	⌘/	Ctrl+/
Generate	⌘N	Alt+Insert
Reformat / optimize imports	⌥⌘L / ^⌥O	Ctrl+Alt+L / O

NAVIGATION & SEARCH (11–12)

ACTION	MACOS	WIN / LINUX
Declaration	⌘B	Ctrl+B
Implementations	⌥⌘B	Ctrl+Alt+B
Super method	⌘U	Ctrl+U
File Structure	⌘F12	Ctrl+F12
Type / Call Hierarchy	^H / ^⌥H	Ctrl+H / Ctrl+Alt+H
Go to Class / File	⌘O / ⇧⌘O	Ctrl+N / Ctrl+Shift+N
Go to Line	⌘L	Ctrl+G
Back / Forward	⌘[/ ⌘]	Ctrl+Alt+← / →
Last edit location	⇧⌘⌫	Ctrl+Shift+Backspace
Find Usages	⌥F7	Alt+F7
Find / Replace in Files	⇧⌘F / ⇧⌘R	Ctrl+Shift+F / R
Bookmark	F3	F11
Go to Test	⇧⌘T	Ctrl+Shift+T
Recent Locations	⇧⌘E	Ctrl+Shift+E

REFACTORING (15–16)

ACTION	MACOS	WIN / LINUX
Rename	⇧F6	Shift+F6
Extract Variable	⌥⌘V	Ctrl+Alt+V
Extract Method	⌥⌘M	Ctrl+Alt+M
Extract Constant / Field	⌥⌘C / ⌥⌘F	Ctrl+Alt+C / F
Change Signature	⌘F6	Ctrl+F6
Move / Inline	F6 / ⌥⌘N	F6 / Ctrl+Alt+N
Refactor This	^T	Ctrl+Alt+Shift+T
Surround With	⌥⌘T	Ctrl+Alt+T

RUN, DEBUG & VCS (18–28)

ACTION	MACOS	WIN / LINUX
Run / Debug	^R / ^D	Shift+F10 / Shift+F9
Run Anything	^^	Ctrl Ctrl
Toggle breakpoint	⌘F8	Ctrl+F8
Step Over / Into / Out	F8 / F7 / ⇧F8	F8 / F7 / Shift+F8
Resume / Evaluate	⌥⌘R / ⌥F8	F9 / Alt+F8
Commit / Push	⌘K / ⇧⌘K	Ctrl+K / Ctrl+Shift+K
Update Project	⌘T	Ctrl+T
VCS popup	^V	Alt+

⚠ COMMON MISTAKES

- Copying VS Code habits: ⌘D here duplicates, ⌥↑ extends selection
- On Windows, `Ctrl+Y` deletes a line — it's not redo
- Assuming a custom keymap matches this sheet — check Settings › Keymap

✓ TIPS & BEST PRACTICES

- Print Help › Keymap Reference for your exact keymap
- Use Find Action when a shortcut slips your mind
- Learn five a week; Key Promoter X helps

QUICK SUMMARY

- ⇧⇧ / ⇧⌘A — find anything
- ⌥⏎ — fix anything
- ⌘B / ⌥F7 — navigate / usages
- ⇧F6 / ⌥⌘M — rename / extract

Sheet 32

Quick Reference: Actions, Templates & Files

Find Action names · live templates · postfix completions · `.idea/` files · `.http` syntax · VM options

MUST-KNOW FIND ACTION NAMES

TYPE IN ⇧⌘A	DOES
Invalidate Caches	fix stale index / red code
Reload All Maven/Gradle	re-sync builds
Local History	restore old versions
Toggle Distraction Free	hide all UI
Run Inspection by Name	one fix across project
Migrate Packages	javax → jakarta
Show Bytecode	javap for current class

TEMPLATES & POSTFIX (06–07)

TYPE + ⇥	EXPANDS TO
psvm / main	main method
sout / soutv	println / with variable
fori / iter	index / for-each loop
ifn / inn	if null / not null
prsf	private static final
.var	declare variable
.for / .fori	loop over expr
.nn / .null	null checks
.try	wrap in try/catch

.IDEA/ FILES (02)

PATH	COMMIT?
.idea/codeStyles/	yes
.idea/inspectionProfiles/	yes
.run/*.run.xml	yes
.idea/misc.xml	usually
.idea/workspace.xml	no
.idea/shelf/	no
*.iml (Maven/Gradle)	no
http-client.private…json	no

.HTTP SYNTAX (26)

SYNTAX	MEANING
###	new request + name
GET {{host}}/x	method + URL
Header: value	headers
blank line + body	request body
> {% … %}	response handler
< ./body.json	body from file
@name = value	file variable

VM OPTIONS (18, 20)

OPTION	USE
-Xmx2g	max heap
-XX:+UseZGC	low-latency GC
-Dkey=value	system property
-agentlib:jdwp=…:5005	remote debug
-XX:+HeapDumpOnOutOfMemoryError	dump on OOM
--enable-preview	preview features

SETTINGS PATHS TO KNOW

SETTINGS ›	FOR
Editor › General › Auto Import	optimize imports on the fly
Tools › Actions on Save	reformat, imports, cleanup
Editor › Code Style › Java	formatting scheme
Editor › Inspections	profiles, severities
Editor › Live Templates	custom templates
Build… › Build Tools › Maven / Gradle	build tool, JVM, delegation
Build… › Compiler › Annotation Processors	Lombok, MapStruct

FREE VS SUBSCRIPTION (01)

FEATURE	TIER
Java, Kotlin, debugger, Git	free
basic database tools	free
Spring, Jakarta EE, app servers	subscription
HTTP Client, full DB tools	subscription
profiler, remote dev	subscription
cloud AI chat / agents	JetBrains AI plan

⚠ COMMON MISTAKES

- Invalidating caches as the first fix — try Reload Maven/Gradle first; reindexing is slow
- Committing `workspace.xml` — conflicts on every pull
- Editing IDE VM options (`Help › Edit Custom VM Options`) when you meant the app's run config

✓ TIPS & BEST PRACTICES

- Analyze Stack Trace turns pasted logs into clickable code links
- Show Bytecode helps explain records, lambdas and switch
- Keep `.http` files beside controllers as living docs

QUICK SUMMARY

- `⇧⌘A` names — hidden power tools
- `psvm / .var` — templates & postfix
- `.idea/` — commit shared parts
- `.http` — runnable requests